
Dimensionality Reduction

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Slides that adapt figures or content from other sources have a link to the source.

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Outline

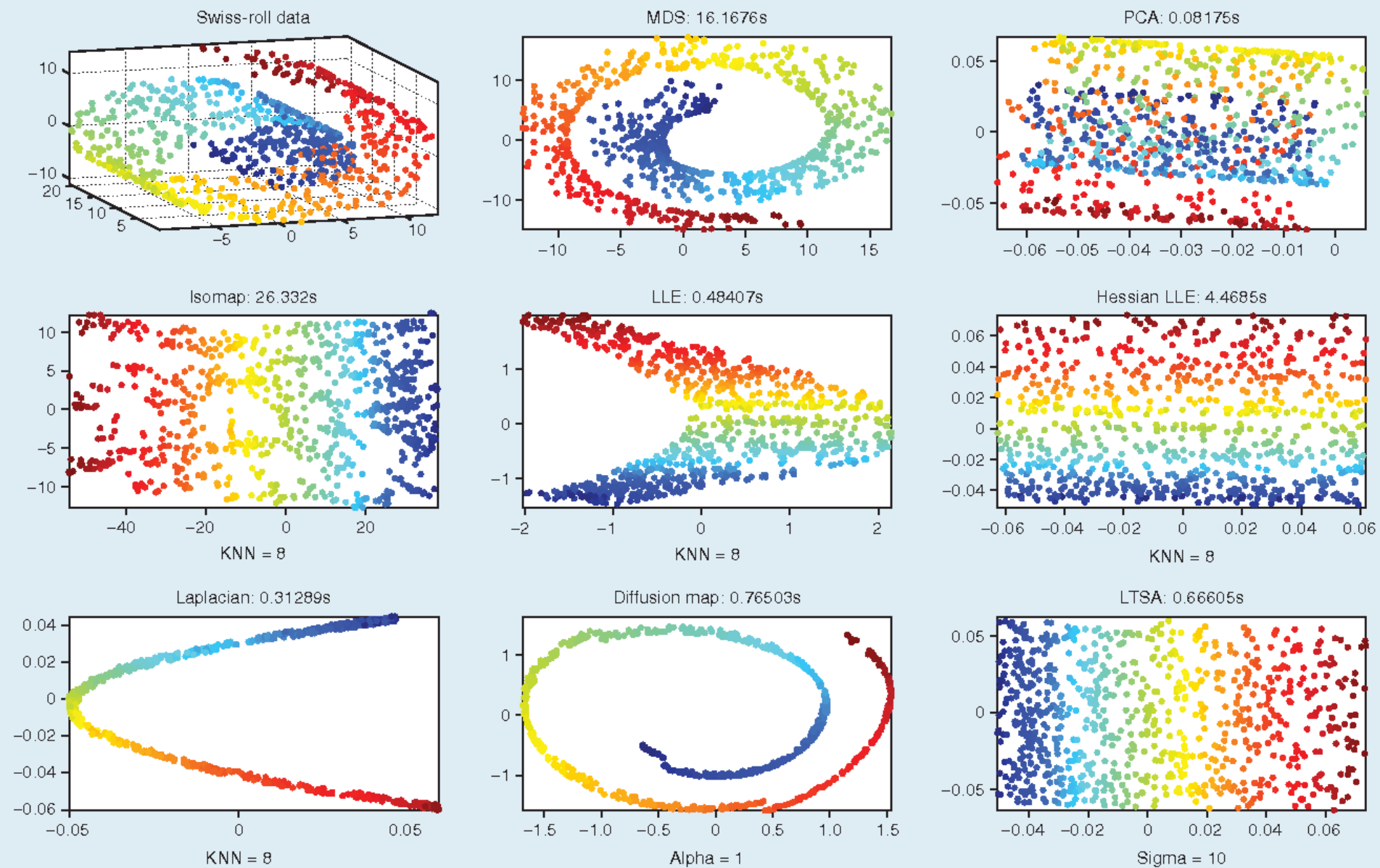
- Dimensionality Reduction Overview
- DR Algorithms:
 - Principal Component Analysis (PCA)
 - Autoencoders
 - t-SNE
 - UMAP

Dimensionality Reduction Overview

- Given an unlabelled dataset set $S = \{x_1, x_2, x_3, \dots, x_m\}$ where x_i has n features (dimensions).
- **Goal:** A lower dimensional mapping of the dataset S from n features to k features where $k < n$.
- There are two general approaches to reduce number of features:
 - **Feature Selection:** from n features, *select* the k most relevant features. Relevance here depends on the task (e.g. prediction, visualization compression). We saw one way of doing this using $L1$ regularization. There are other ways to do this, but we won't discuss it.
 - **Feature Extraction:** *derive* k features from the n features. That is, the k features are computed as a formula of the n features. We will discuss some methods that do this.

Dimensionality Reduction Use-Cases

- There are number of use-cases and benefits to reducing the dimension:
 - In supervised learning, it reduces the runtime of training. Fewer features, means fewer parameters for fitting.
 - In supervised learning, it reduces the complexity of the model (combats overfitting). Reducing features is a form of regularization.
 - In supervised learning, it can remove noisy or irrelevant features which can help in getting a better model.
 - Visualization; can reduce a high dimension dataset to 2D or 3D and plot.
 - Data compression which helps save storage space.



3D Swiss-roll data (top-left) is reduced to 2D using eight dimensionality reduction techniques.

Image from: <https://ieeexplore.ieee.org/document/5401149>

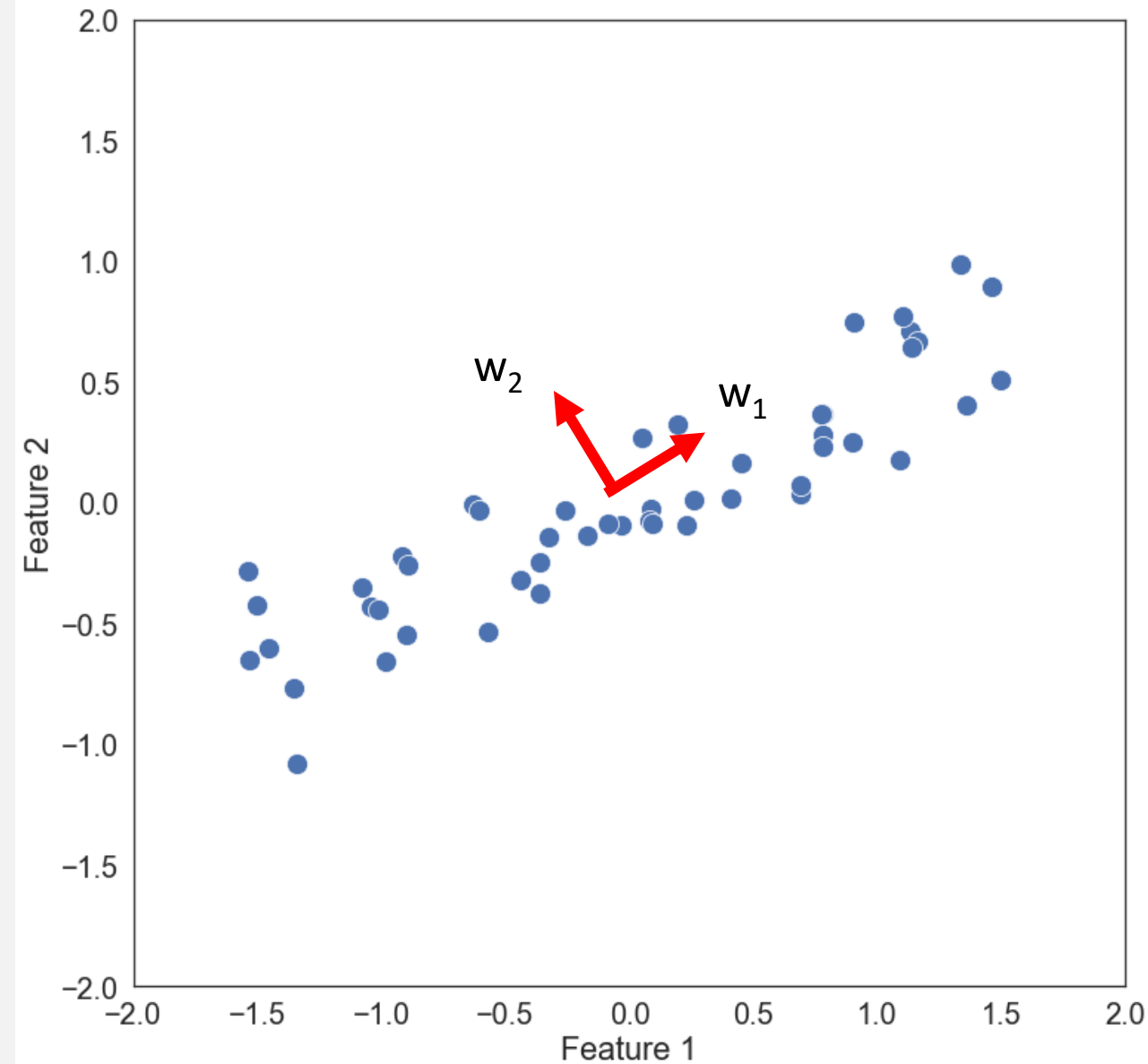
Dimensionality Reduction: Feature Extraction methods

- **Feature Extraction:** *derive* k features from the n features. That is, the k features are computed as a formula of the n features. We will discuss some methods that do this.
- In general, feature extraction methods *transform* or *project* the n dimensional data to a k dimensional subspace.
- They can be divided into linear projection methods and non-linear projection methods.
- In general, these methods will make assumptions and optimize some metric. They will attempt to find the best projection of the data from n dimensions to k dimensions that optimize their defined metric.
- We'll discuss four techniques: PCA (linear), Autoencoders (non-linear), t-SNE (non-linear), and UMAP (non-linear).

Principal Component Analysis (PCA)

PCA Overview [1]

- Principal component analysis (PCA) is a linear projection of datapoints into a lower dimensional subspace such that the *variance of the projected data is maximized*.
- Consider the shown 2-feature dataset.
- The principal components of this 2-feature data are the 2 vectors w_1, w_2 that maximize the variance of the projected data onto these 2 vectors.
- Two questions here:
 - What do we mean by projection?
 - What do we mean by variance?



Linear Algebra Concept – Vector Projection

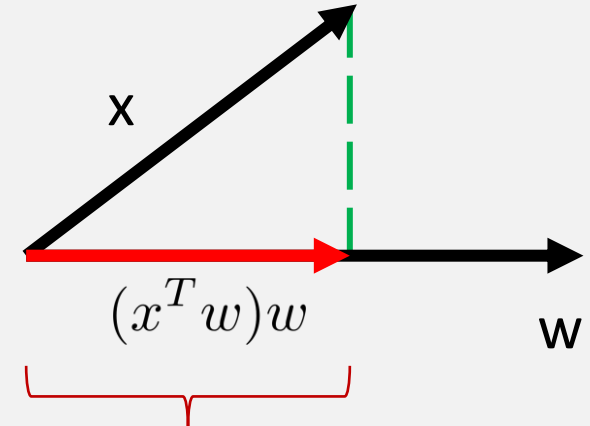
- Let w be a **unit** vector (magnitude 1). Then the projection of a vector x **onto** w is given by:

$$Proj_w x = \underbrace{(x^T w)}_{\text{scalar value}} w$$

Scalar value that indicates the direction and length (magnitude) of projection of x along the vector w



Note here that $(x^T w)$ is a negative scalar.



This red vector is the projection. Its length is $(x^T w)$.

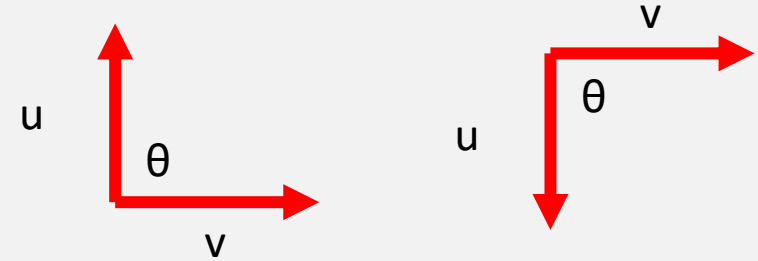


Note here that $(x^T w)$ is zero.

Linear Algebra Concept – Orthogonal and Orthonormal

- Two vectors u and v are said to be **orthogonal** if they are perpendicular to each other. In other words, if their dot product is zero:

$$u^T v = ||u||_2 ||v||_2 \cos(\theta) = 0$$

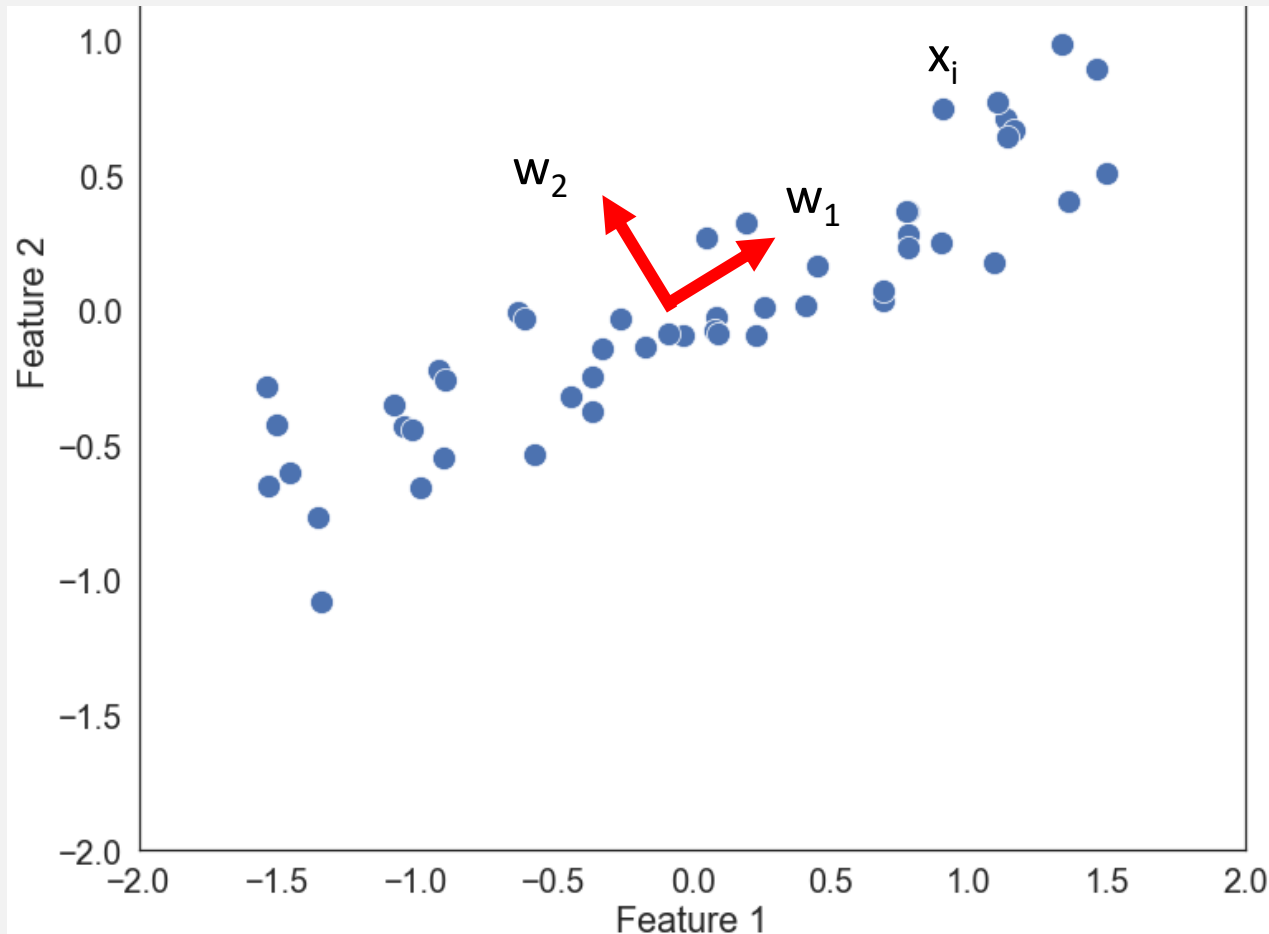


- Two vectors u and v are said to be **orthonormal** if they are orthogonal (perpendicular) and are unit vectors (magnitude is 1).
- A set of vectors $\{w_1, w_2, \dots, w_n\}$ are said to be orthonormal, if they are all unit vectors and are all orthogonal to each other.

$$w_i^T w_j = 0 \text{ for all } i \neq j$$

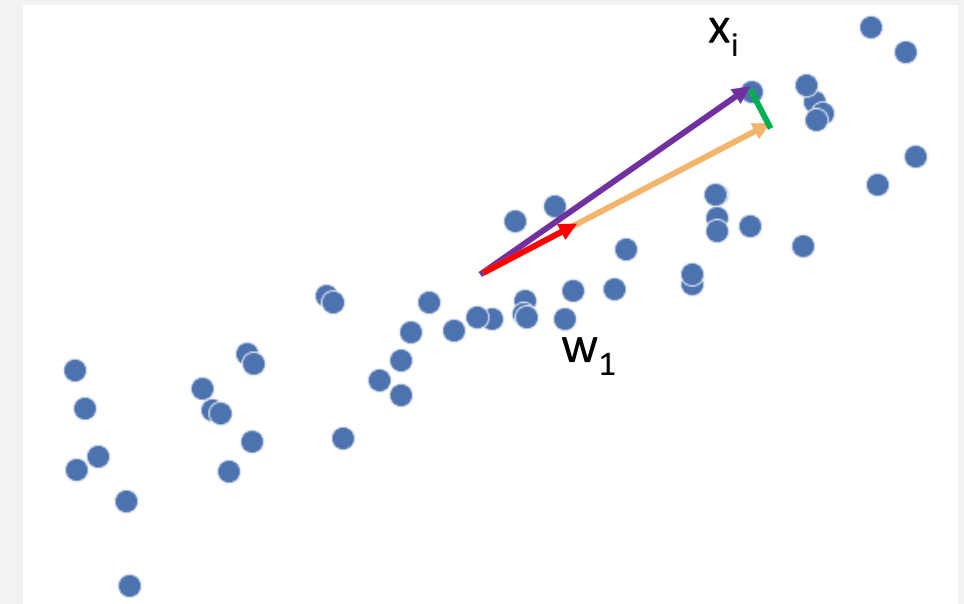
Projecting onto Principal Components [1]

- The principal components of a dataset with n features are n orthonormal vectors.
- We can project the datapoints (treated as vectors) onto a principal component.



$$Proj_{w_1} x_i = \underbrace{(x_i^T w_1)}_{\text{Scalar value}} w_1$$

Scalar value that indicates the length (magnitude) of projection of x_i along the principal component w_1



Projecting onto Principal Components [2]

- The principal components of a dataset with n features are n orthonormal vectors.
- We can project the datapoints (treated as vectors) onto a principal component.
- Suppose $x_i = [0.9, 0.75]$ and the PCs are $w_1 = [0.8, 0.6]$ and $w_2 = [-0.6, 0.8]$

$$\begin{aligned} Proj_{w_1} x_i &= (x_i^T w_1) w_1 \\ &= \underbrace{1.17}_{\text{scalar value}} [0.8, 0.6] \end{aligned}$$

Scalar value that indicates the length (magnitude) of projection of x_i along the principal component w_1

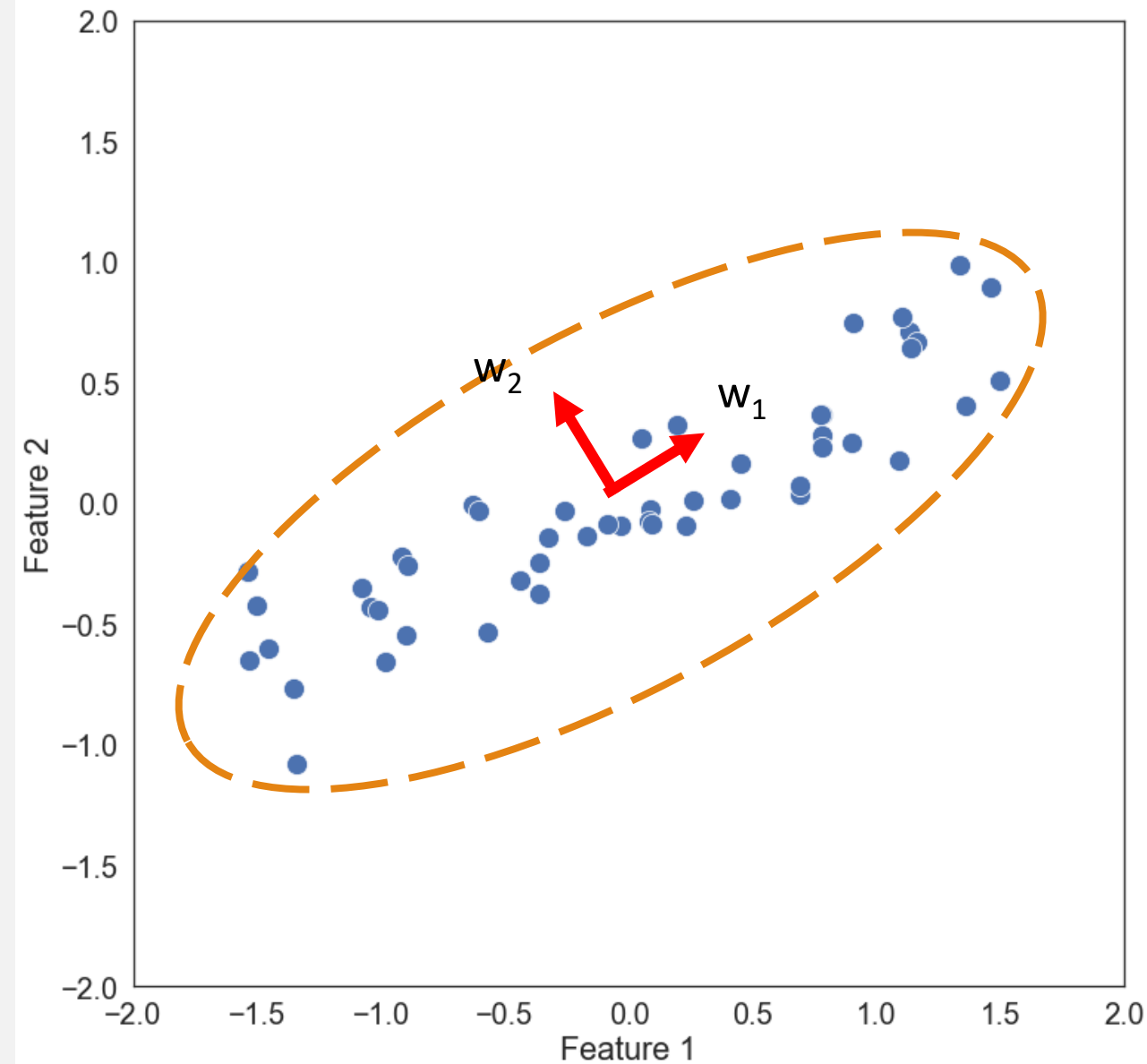
$$\begin{aligned} Proj_{w_2} x_i &= (x_i^T w_2) w_2 \\ &= \underbrace{0.06}_{\text{scalar value}} [-0.6, 0.8] \end{aligned}$$

Scalar value that indicates the length (magnitude) of projection of x_i along the principal component w_2

- If we wanted to project this data using PCA down to 1-dimension, we would use the first PC, and $z_i = [1.17]$ would be the datapoint x_i in this lower 1-dimension.
- What about if we used 2 PCs? $z_i = [1.17, 0.06]$. Not helpful if we want reduce dimensions.
- Sidenote: we can reconstruct x_i as $x_i = (x_i^T w_1) w_1 + (x_i^T w_2) w_2$.

What do we mean by variance of the data?

- In the figure, the PCs are the 2 vectors that are **orthonormal** and explain the **spread** (variance) of the data.
- Notice that the data has bigger spread (variance) along w_1 than along w_2 .
- For this data, the **first** principal component is the vector w_1 since it explains **most** of the variance in the data.
- The **second** principal component for this data is w_2 since it explains the **second most** variance in the data.
- For n -dimension data, the n principal components are the n **orthonormal** vectors $w_1, w_2, \dots, w_n$ that explain most of the variance in the data in **sequential** order of variance.



Using PCA

- For n -dimension data, the n principal components are the n **orthonormal** vectors $w_1, w_2, \dots, w_n$ that explain most of the variance in the data in **sequential** order of variance.
- Assume for now that given dataset in n -dimensions, you are able to get the n PC vectors.
- Suppose that you want to use PCA to reduce to k dimensions. For this, you will consider the top- k principal components in terms of variance explained.
- This would be the k first PCs: $w_1, w_2, \dots, w_k$. Project the data onto these k vectors and grab the scalars.

$$Proj_{w_1} x_i = (x_i^T w_1) w_1 \quad Proj_{w_2} x_i = (x_i^T w_2) w_2 \quad \dots \quad Proj_{w_k} x_i = (x_i^T w_k) w_k$$

- These k scalars will be your lower dimensional representation of a datapoint.
- We can do this projection in one go using matrix projection.

Linear Algebra Concepts – Projecting onto multiple vectors

- Let $\{w_1, w_2, \dots, w_k\}$ be k orthonormal vectors in $\mathbb{R}^n$. Then the projection of a vector $x \in \mathbb{R}^n$ onto $W = [w_1, w_2, \dots, w_k]$ where $W \in \mathbb{R}^{n \times k}$ is given by:

$$\text{Proj}_W x_i = W(W^T x) = W \begin{bmatrix} w_1^T x \\ w_2^T x \\ \vdots \\ w_k^T x \end{bmatrix} = \begin{bmatrix} | & | & \dots & | \\ w_1 & w_2 & \dots & w_k \\ | & | & \dots & | \end{bmatrix} \begin{bmatrix} w_1^T x \\ w_2^T x \\ \vdots \\ w_k^T x \end{bmatrix} = \sum_{j=1}^k (x_i^T w_j) w_j$$

This is the k dimensional vector that represents the reduction of the datapoint x_i to k dimensions.

Sidenote: What is this doing? If the vectors are the PCs of the dataset, then this reconstructs x_i as best it can using k PCs.

Using PCA

- Thus, for n -dimension data, suppose the n principal components are given by $w_1, w_2, \dots, w_n$. We can project a datapoint x_i to lower dimension k using $W = [w_1, w_2, \dots, w_k]$:

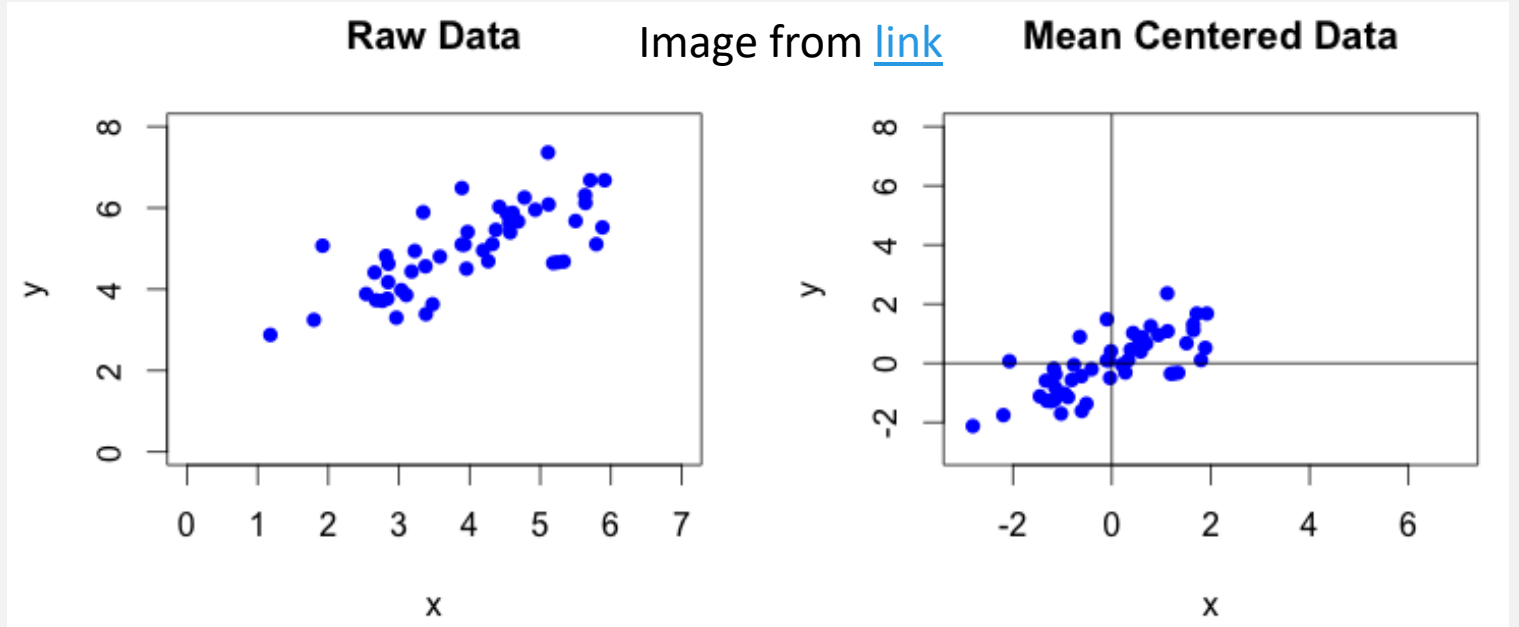
$$z_i = W^T x_i = \begin{bmatrix} w_1^T x_i \\ w_2^T x_i \\ \vdots \\ w_k^T x_i \end{bmatrix}$$

Using PCA in practice with a dataset

- Suppose for now that you have a method $PCA(X)$ that allows you to calculate the k principal components of a dataset.
- For a given data matrix X of m instances with n features, you should first centralize X by subtracting the feature (column) means.

$$x_i := x_i - \mu_{data}$$

- This is done so that the calculated PC vectors are relative to the center of your dataset.



- With the centered data X_c , you can call $PCA(X_c)$ to get $W = [w_1, w_2, \dots, w_k]$. Then reduce to lower dimensions as follows:

$$Z = X_c W \quad \text{where } X_c \in \mathbb{R}^{m \times n}, W \in \mathbb{R}^{n \times k}$$

PCA Example with 3-features

- The following showcases the projection of datapoints from 3D to 2D. The result is a projection onto a plane. The first two principal components were used for this projection.

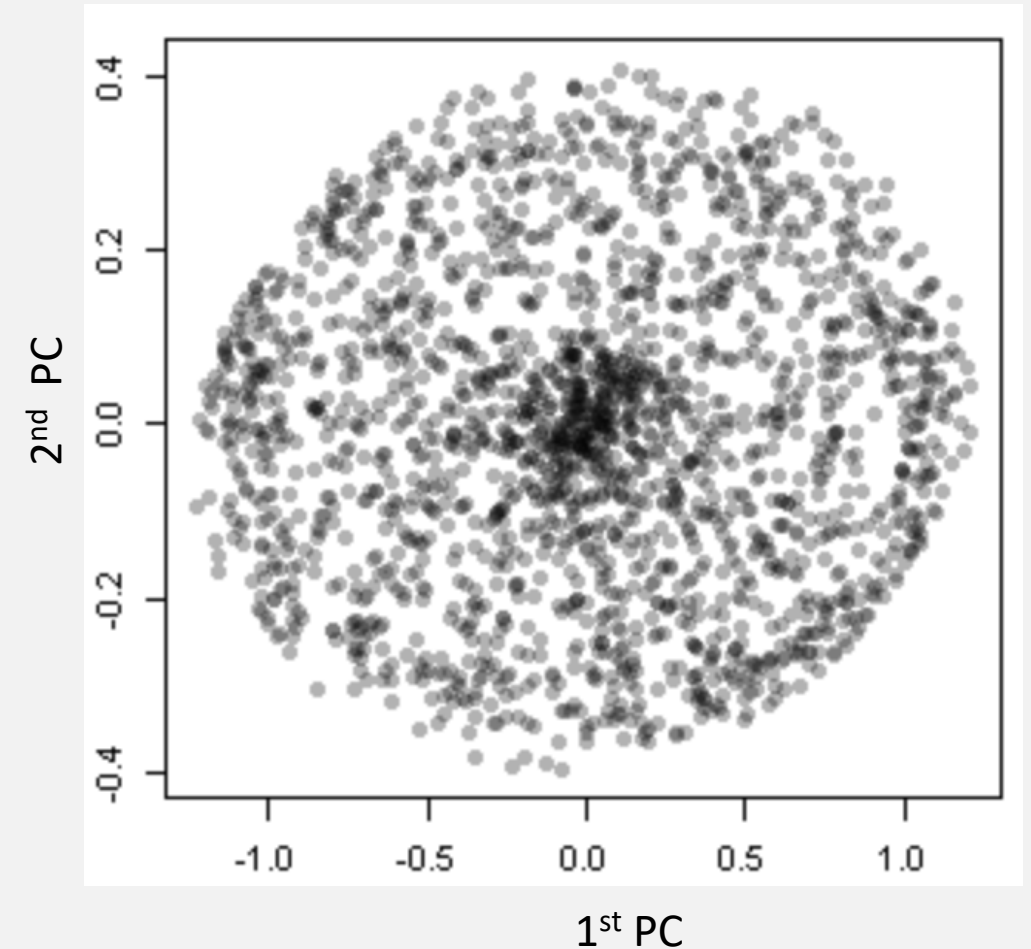
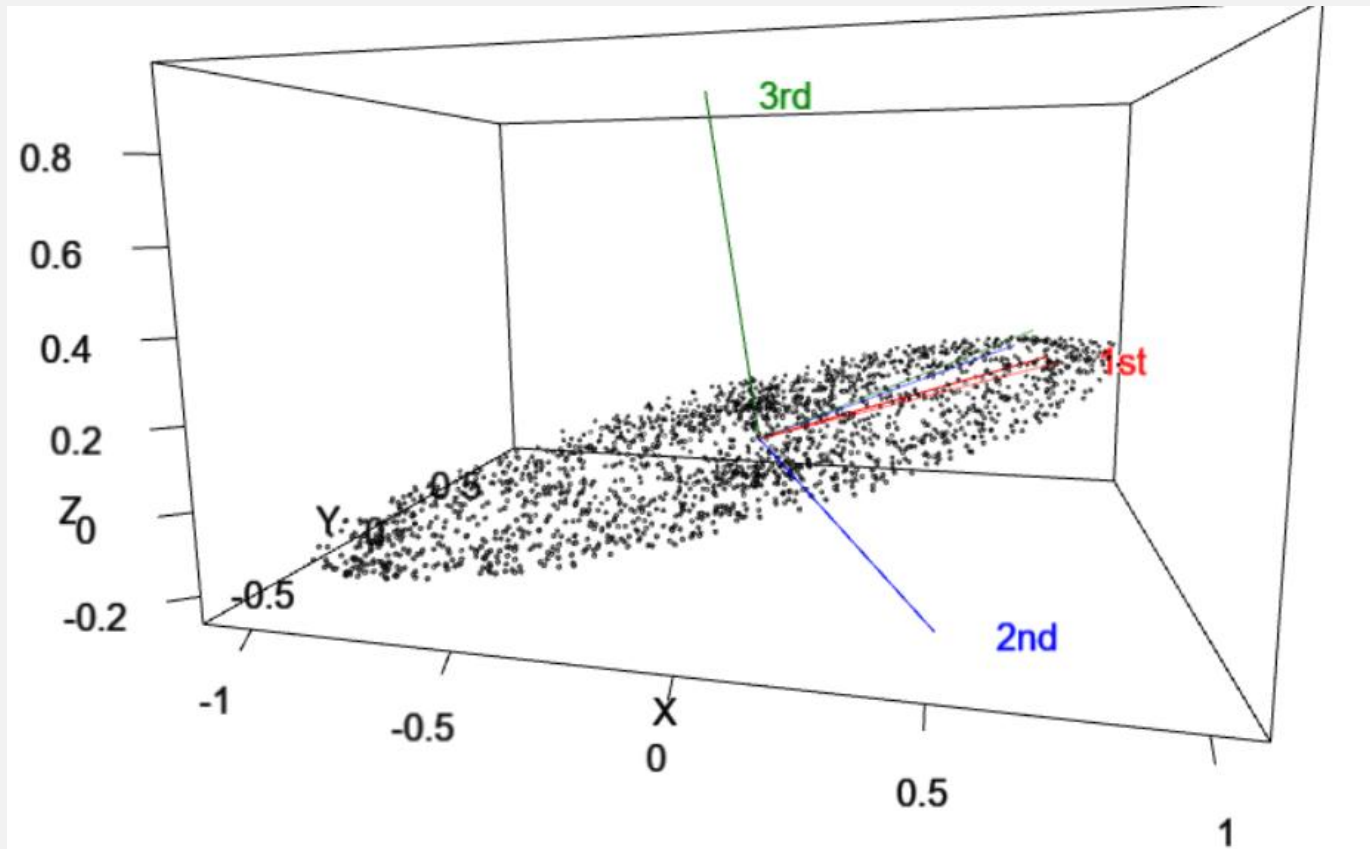


Image from: [link](#)

PCA Example with 3-features

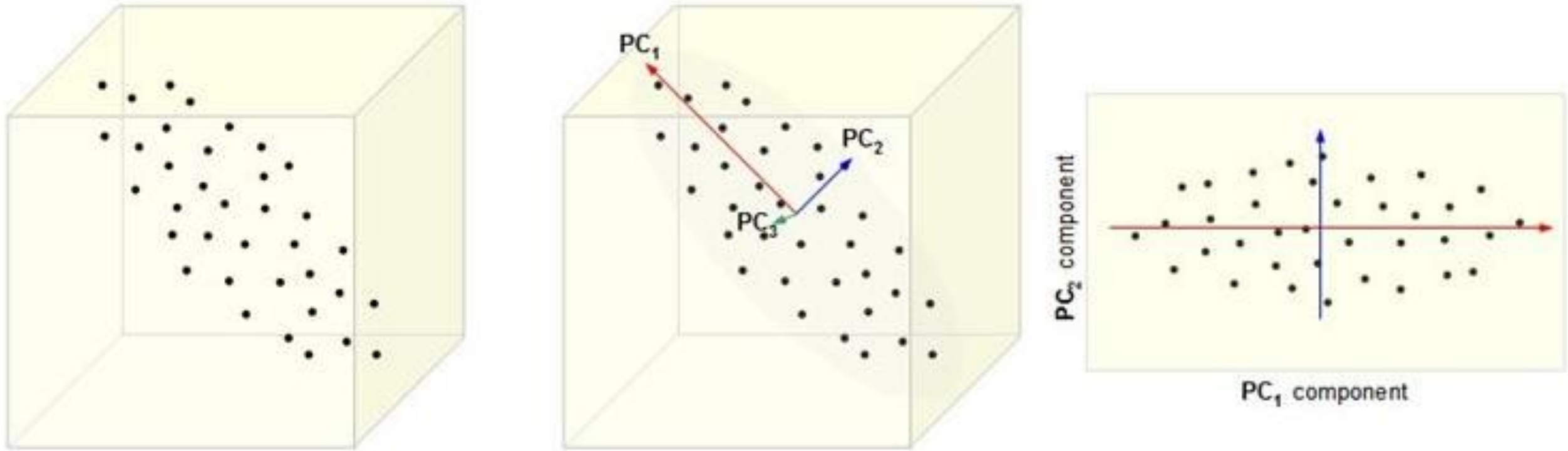
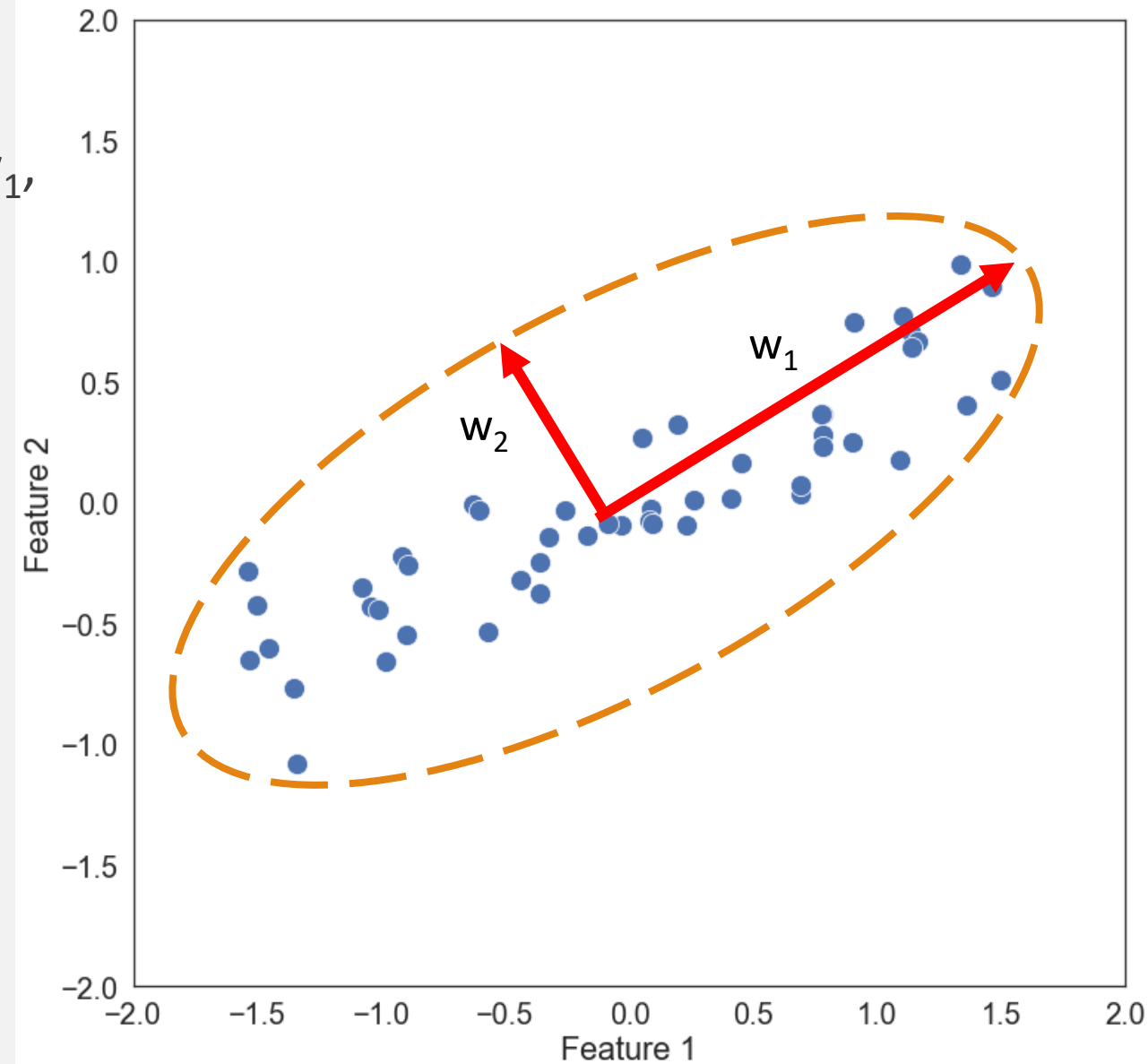


Image from: [link](#)

Using PCA [1]

- Thus, for n -dimension data, the n principal components are the n **orthonormal** vectors $w_1, w_2, \dots, w_n$ that explain most of the variance in the data in sequential order of variance.
- Thus, PCA considers the top- k principal components in terms of variance explained. The data is then **projected onto** the top- k principal components which gives k -dimensions.
- So, for this 2-feature dataset, assume for now we can find the PC vectors w_1 and w_2 . How does PCA project this data to one dimension?



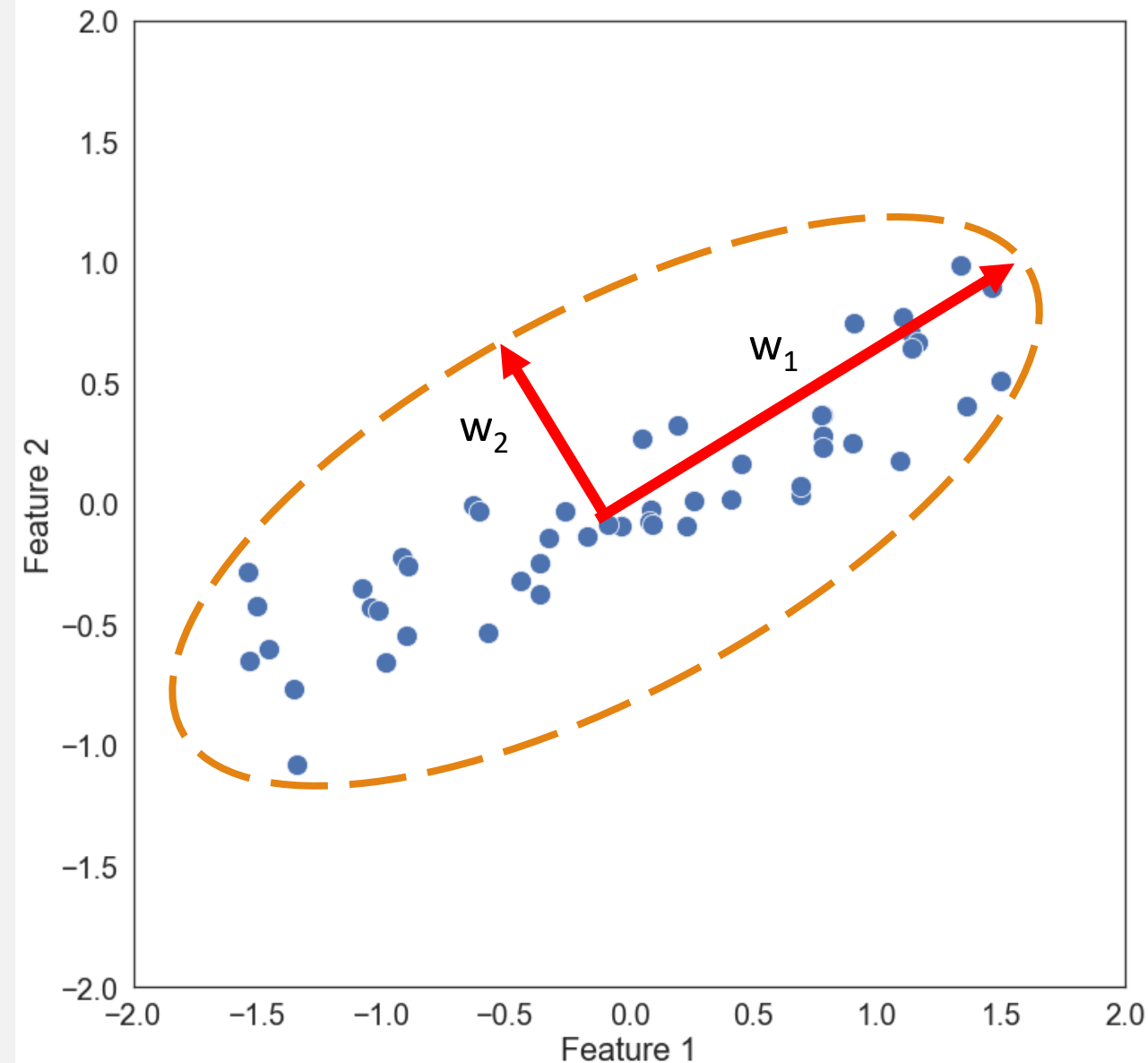
PCA Overview [4]

- So, for this 2-feature dataset, assume for now we can find the PC vectors w_1 and w_2 . How does PCA this data to one dimension?
- The projection of a point x_i onto w_1 is:

$$Proj_{w_1} x_i = \underbrace{(x_i^T w_1)}_{\text{Scalar value that indicates the length (magnitude) of projection of } x_i \text{ along the principal component } w_1} w_1$$

Scalar value that indicates the length (magnitude) of projection of x_i along the principal component w_1

- The value $(x_i^T w_1)$ is the one-dimension reduction of x_i
- Thus, the reduction is $Z = \{z_1, z_2, \dots, z_m\}$ where $z_i = (x_i^T w_1)$



How to get the principal components of a dataset? [1]

- As mentioned, assume we have subtracted the feature (column) means from the data. That is, we centered the data around the origin in n -dimensions.

$$x_i := x_i - \mu_{data}$$

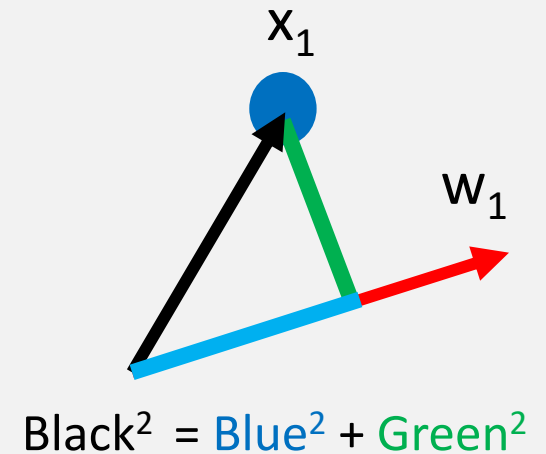
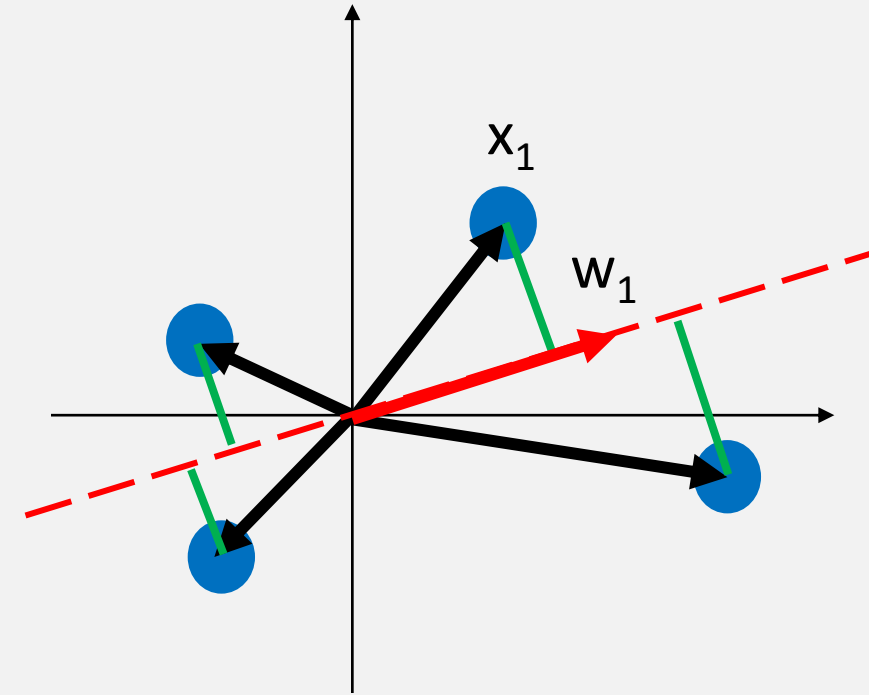
- How to compute the principal components?
- One formulation is to find the k orthonormal vectors such that if we project the datapoints onto these k vectors, then it will give the **maximum** variance for each projection.

Principal Components [2]

- Consider the 2-feature dataset. The data has 4 points centered around the origin.
- We want to find a vector w_1 such that if we project the blue points onto w_1 , the variance will be maximized.
- The variance here is the **blue line** length (magnitude) of the projection relative to the origin. This is the absolute value of the scalar: $(x^T w)$
- We want to choose w_1 such that it will maximize the absolute scalar in the projection for ALL points in the dataset.
- Thus, we want to solve the following optimization:

$$\arg \max_{\|w_1\|=1} \sum_{i=1}^m \underbrace{(x_i^T w_1)^2}_{\text{Maximize the absolute scalar in the projection}} = \arg \max_{\|w_1\|=1} \|X w_1\|_2^2$$

Maximize the absolute scalar in the projection



How to get the principal components of a dataset? [1]

- There are two formulations for PCA that both have the same solution, i.e. the principal components of the data.
- As mentioned, assume we have subtracted the feature (column) means from the data. That is, we centered the data around the origin in n -dimensions.

$$x_i := x_i - \mu_{data}$$

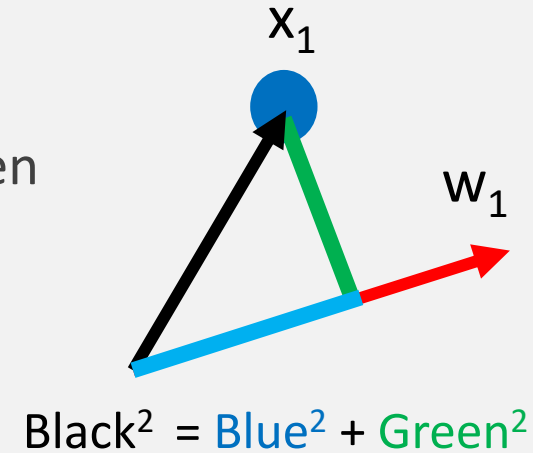
- One formulation is to find the k orthonormal vectors such that if we project the datapoints onto these k vectors, then it will give the **maximum** variance for each projection.
- Another formulation is to find the k orthonormal vectors such that the projection of the data onto these orthonormal vectors **minimizes** the euclidean distance between the original points and the projections.

Principal Components [3]

$$\arg \max_{\|w_1\|=1} w_1^T X^T X w_1$$

- Another formulation is to manipulate the optimization into a minimization of distances.
- Notice that the datapoints are fixed, and we can only play around with w_1 .
- Using pythagoras theorem, we have that the **variance (blue line)** is maximized when the distance between the point and the projection is minimized (**green line**).

$$\arg \max_{\|w_1\|=1} \underbrace{\sum_{i=1}^m (x_i^T w_1)^2}_{\text{Maximize Blue line (variance)}} = \arg \min_{\|w_1\|=1} \underbrace{\sum_{i=1}^m \|x_i - Proj_{w_1}(x_i)\|^2}_{\text{Minimize Green line (distance)}}$$



- Solving this can be achieved by the singular-value-decomposition (SVD) of the dataset matrix X .

Principal Components [4]

$$\begin{aligned}\arg \min_{\|w_1\|=1} \sum_{i=1}^m \|x_i - (x_i^T w_1) w_1\|^2 &= \arg \min_{\|w_1\|=1} \sum_{i=1}^m \|x_i - w_1 (w_1^T x_i)\|^2 \\ &= \arg \min_{\|w_1\|=1} \sum_{i=1}^m \|x_i - (w_1 w_1^T) x_i\|^2 \\ &= \arg \min_{\|w_1\|=1} \sum_{i=1}^m \|(I - w_1 w_1^T) x_i\|^2\end{aligned}$$

Expand and simplify

$$= \arg \min_{\|w_1\|=1} \underbrace{\sum_{i=1}^m x_i^T x_i}_{\text{This sum is fixed, it is the data}} - \underbrace{\sum_{i=1}^m w_1^T (x_i x_i^T) w_1}_{\text{This sum is what can be changed by modifying } w}$$

This sum is fixed, it
is the data

This sum is what can be changed
by modifying w

Principal Components [6]

$$\arg \min_{\|w_1\|=1} \underbrace{\sum_{i=1}^m x_i^T x_i}_{\text{This sum is fixed, it is the data}} - \underbrace{\sum_{i=1}^m w_1^T (x_i x_i^T) w_1}_{\text{This sum is what can be changed by modifying } w}$$

This sum is fixed, it
is the data

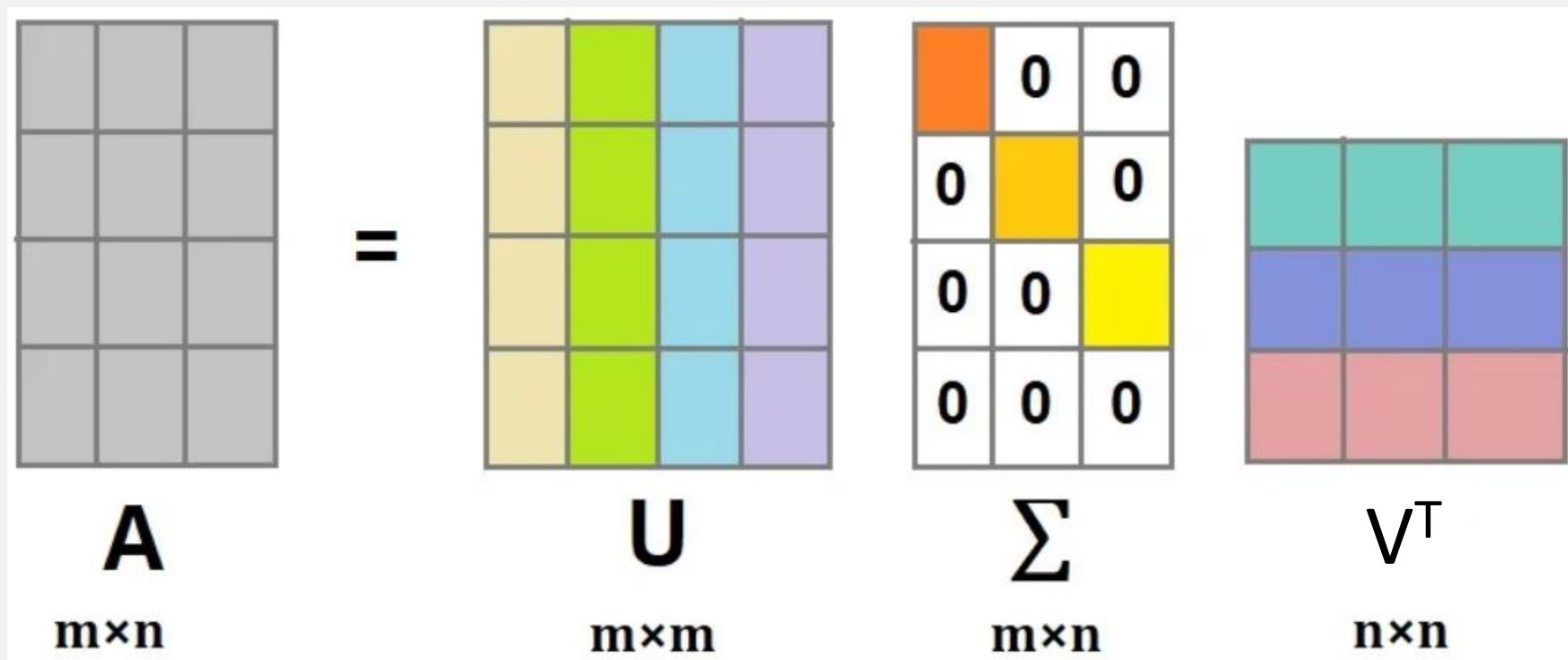
This sum is what can be changed
by modifying w

- Since the first summation term is fixed by the data, then minimizing this entire quantity is achieved by maximizing the second summation term.

$$\arg \max_{\|w_1\|=1} \sum_{i=1}^m w_1^T (x_i x_i^T) w_1 = \arg \max_{\|w_1\|=1} \|X w_1\|^2$$

- Interestingly, we ended up with the same variance maximization formulation. Anyway, how do we solve this?

- **Singular Value Decomposition (SVD):** Any matrix $A \in \mathbb{R}^{m \times n}$ can be decomposed into $A = U\Sigma V^T$.



- $U \in \mathbb{R}^{m \times m}$ consists of m orthonormal columns called the **left singular vectors**.
- $V \in \mathbb{R}^{n \times n}$ consists of n orthonormal columns called the **right singular vectors**.
- $\Sigma \in \mathbb{R}^{m \times n}$ consists of $\min(m, n)$ **singular values** along the diagonal (σ_i).

- In essence, the SVD of a matrix provides us with information on how to reconstruct a matrix.
- The singular values $\sigma_1, \sigma_2, \dots, \sigma_{\min(m,n)}$ along the diagonal of Σ are ordered from largest to smallest. This gives an ordering of importance of the left and right singular vectors in the reconstruction of the matrix.
- The 2-norm of a matrix A is defined as: $\|A\|_2 = \max_{\|w\|=1} \|Aw\|_2 = \sigma_1$
- That is, the 2-norm of a matrix A is the first singular value σ_1 in the SVD of $A = U\Sigma V^T$
- This max is achieved when $w = v_1$, the first right singular vector (column) of V .
- This looks similar to the optimization we are doing in PCA. $\arg \max_{\|w_1\|=1} \|Xw_1\|_2^2$

Principal Components [8]

$$\arg \min_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \sum_{i=1}^m \|x_i - Proj_W(x_i)\|^2 = \arg \min \sum_{i=1}^m \|x_i - WW^T x_i\|^2$$

See [slide](#)

$$= \arg \min \sum_{i=1}^m \|x_i(I - WW^T)\|^2$$

Expand and simplify

$$= \arg \min \sum_{i=1}^m x_i^T x_i - \underbrace{\sum_{i=1}^m x_i^T WW^T x_i}$$

So again, here we want to maximize the second summation term

Principal Components [9]

$$\arg \min_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \sum_{i=1}^m \|x_i - Proj_W(x_i)\|^2 = \arg \min \sum_{i=1}^m \|x_i - WW^T x_i\|^2$$

See [slide](#)

$$= \arg \min \sum_{i=1}^m \|x_i(I - WW^T)\|^2$$

Expand and simplify

$$= \arg \min \sum_{i=1}^m x_i^T x_i - \underbrace{\sum_{i=1}^m x_i^T WW^T x_i}$$

So again, here we want to maximize the second summation term

Principal Components [9]

- From linear algebra, the trace of a square matrix is the sum of the diagonal elements:

$$\text{trace}[A] = \sum_i a_{ii}$$

$$\arg \max \sum_{i=1}^m x_i^T W W^T x_i = \arg \max \sum_{i=1}^m \text{trace}[W^T (x_i x_i^T) W]$$

$$= \arg \max \text{trace}[W^T \sum_{i=1}^m (x_i x_i^T) W]$$

$$= \arg \max \text{trace}[W^T (X^T X) W]$$

$$= \arg \max \|XW\|_F^2$$

- From linear algebra, the Frobenius norm of a matrix is:

$$\begin{aligned} \|A\|_F^2 &= \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2 \\ &= \text{trace}(A^T A) \end{aligned}$$

$$\text{Frobenious norm: } \|A\|_F^2 = \sum_{i=1}^m \sum_{j=1}^n |a_{ij}|^2$$

$$\arg \min_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \sum_{i=1}^m \|x_i - \text{Proj}_W(x_i)\|^2 = \arg \min_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \sum_{i=1}^m \|x_i - WW^T x_i\|^2 = \arg \max_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \|XW\|_F^2$$

- The solution to this are the first k right singular vectors in the SVD decomposition.
- That is, compute the SVD of $X = U\Sigma V^T$, where $V = \{v_1, v_2, \dots, v_n\}$ are the orthonormal right singular vectors. Then the solution is $W = \{v_1, v_2, \dots, v_k\}$. These are the top- k principal components.
- Thus, we can reduce the data to k dimensions using $z_i = (W^T x_i)$ where $W = V[:, :k]$ (the first k columns of V).

$$\arg \max_{\|w_1\|=1} \|Xw_1\|_2^2$$

- Compute the SVD of $X = U\Sigma V^T$, where $V = \{v_1, v_2, \dots, v_n\}$ are the orthonormal right singular vectors. Then the first principal component (solution) is given by v_1 .
- Thus, we can reduce the data to one dimension using $z_i = (x_i^T v_1)$
- What if we want to reduce the data to k dimensions? How do we get the top- k principal components?
- In this case, we want k orthonormal vectors $W = \{w_1, w_2, \dots, w_k\}$ that minimize the distance between the projected points and the original points.

$$\arg \min_{\substack{\{w_1, w_2, \dots, w_k\} \\ w_i^T w_j = 0 \\ \|w_i\|=1}} \sum_{i=1}^n \|Xw_i\|_2^2$$

Principal Components using SVD

$$\arg \min_{\substack{W \in \mathbb{R}^{n \times k} \\ w_i^T w_j = 0 \\ \|w_i\| = 1}} \sum_{i=1}^n \|X w_i\|_2^2$$

- The solution to this are the first k right singular vectors in the SVD decomposition.
- That is, compute the SVD of $X = U\Sigma V^T$, where $V = \{v_1, v_2, \dots, v_n\}$ are the orthonormal right singular vectors. Then the solution is $W = \{v_1, v_2, \dots, v_k\}$. These are the top- k principal components.
- Thus, we can reduce the data to k dimensions using $z_i = (W^T x_i)$ where $W = V[:, :k]$ (the first k columns of V).

Summary of PCA [1]

- Given a dataset $\{x_1, x_2, \dots, x_m\}$ where $x_i \in \mathbb{R}^n$ has n features. Let the data matrix be $X \in \mathbb{R}^{m \times n}$. We would like to reduce this data to k dimensions.
- Mean normalize the data matrix X . That is, $X := X - X.mean(axis = 0)$
- The k principal components of X are the k orthonormal vectors $w_1, w_2, \dots, w_k$ that maximize the variance of the projected data.
- The solution to this are the first k right singular vectors in the SVD of X .
- That is, compute the SVD of $X = U\Sigma V^T$, where $V = \{v_1, v_2, \dots, v_n\}$ are the orthonormal right singular vectors. Then the solution is $W = \{v_1, v_2, \dots, v_k\}$. These are the top- k principal components.
- Thus, we can reduce the data to k dimensions using $z_i = V[:, : k]^T x_i$. The whole data can be transformed by $Z = XV[:, : k]$

PCA Implementation

- [Scikit-learn implementation](#)

```
1 from sklearn.decomposition import PCA
2
3 pca = PCA(n_components=2)
4
5 pca.fit(X)          # computes principal components using SVD
6
7 pca.transform(X)    # computes  $X @ V[:, :k]$ 
```

- Can also use numpy to compute the SVD and project.

```
1 U, S, V = np.linalg.svd(X, full_matrices=True)
2 V = V.T      # note that numpy returns V transposed
3
4 X @ V[:, :2] # project onto the two first principal components
```

How to set k ? [1]

- For a dataset X in n dimensions, how to choose the reduction to k dimensions to preserve as much of the data as possible?
- For visualization in 2D and 3D, you are forced to set this.
- For data compression, then recall that the principal components explain the variance in the data in sequential order.
- These variances are the square of the singular values in the Σ matrix.

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ 0 & \vdots & \dots & 0 \\ 0 & 0 & \dots & \sigma_n \end{bmatrix}$$

How to set k ? [2]

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \dots & 0 \\ 0 & \sigma_2 & \dots & 0 \\ 0 & \vdots & \dots & 0 \\ 0 & 0 & \dots & \sigma_n \end{bmatrix}$$

- The % variance explained by the first principal component is $\frac{\sigma_1^2}{\sum \sigma_i^2}$. The % variance explained by the second principal component is $\frac{\sigma_2^2}{\sum \sigma_i^2}$. And so on for the remaining components.
- It follows that the variance explained by projecting onto the top- k principal components:

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_i \sigma_i^2}$$

- This ratio is 1 when we include ALL components.
- A common rule is to set k such that this ratio is at least 0.90 (or 0.95). That is, 90% of the variance in the data is explained by the top- k principal components.

How to set k ? [3]

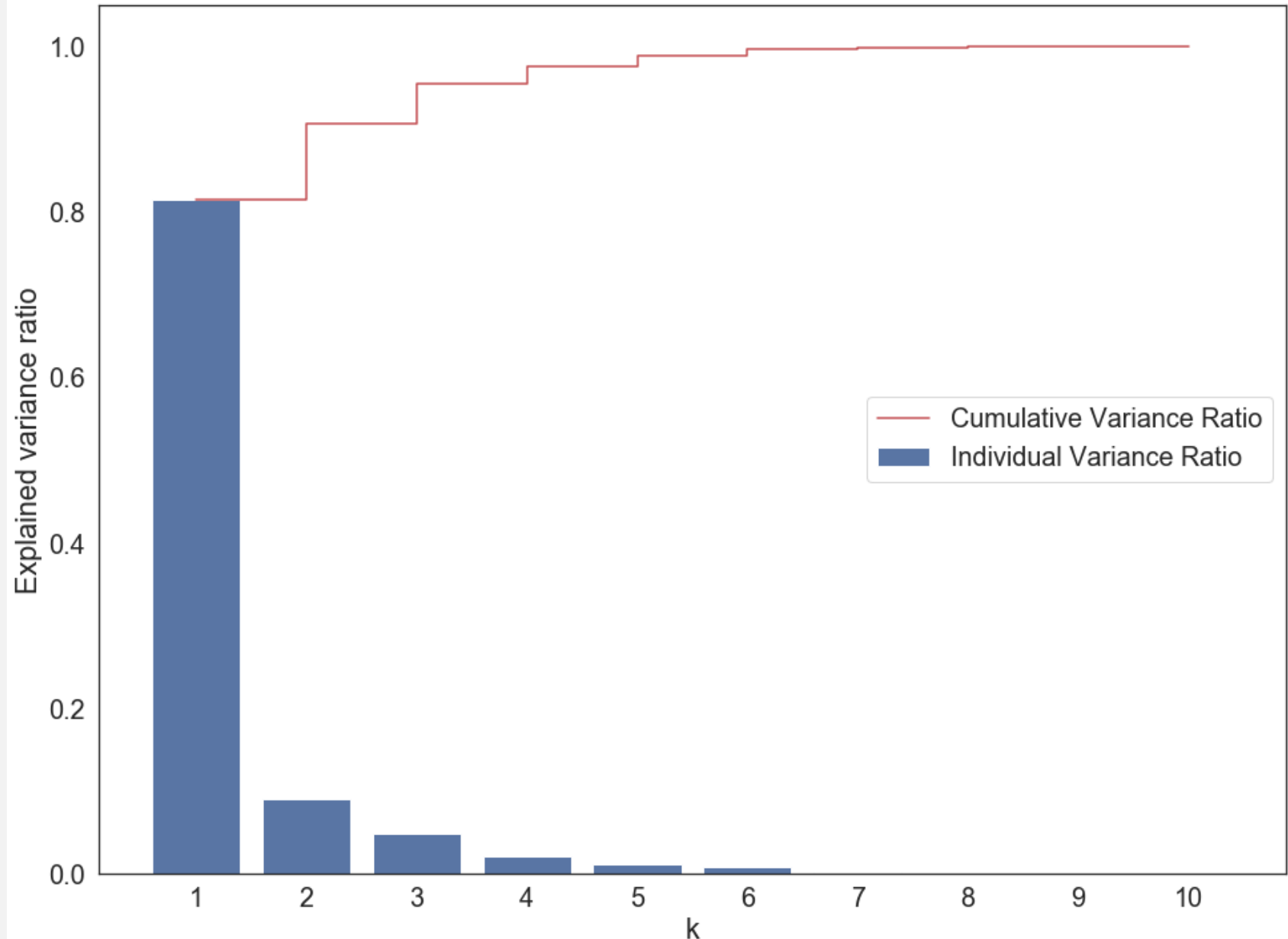
- A common rule is to set k such that this ratio is at least 0.90 (or 0.95). That is, 90% of the variance in the data is explained by the top- k principal components.

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_i \sigma_i^2} \geq 0.9$$

```
1 U, S, V = np.linalg.svd(X, full_matrices=True)
2 V = V.T
3
4 total_var = (S**2).sum()
5 for k in range(1, S.shape[0]+1):
6     var_ratio = (S[:k]**2).sum() / total_var
7     print('For k={}, var_ratio={}'.format(k, var_ratio))
```

How to set k ? [4]

- Can also visualize this ratio as we increase k in a plot.
- We see here that we can use $k = 3$ which explains 95% of the variance in the data.



PCA Example [1]

- Consider the following dataset (already mean-centered) and its SVD decomposition.

$X =$

feat1	feat2	feat3	feat4	feat5
0.33	-0.20	0.22	0.04	-0.14
-0.26	0.45	-0.36	0.05	0.39
-0.13	-0.09	0.17	0.45	-0.12
0.46	-0.05	-0.25	-0.20	0.20
0.09	0.07	-0.28	-0.02	0.07
-0.49	-0.16	0.48	-0.33	-0.39

$$X \in \mathbb{R}^{6 \times 5}$$

$$U \in \mathbb{R}^{6 \times 6}$$

$$\Sigma \in \mathbb{R}^{6 \times 5}$$

$$V \in \mathbb{R}^{5 \times 5}$$

$U =$

0.1534	0.5191	-0.1398	0.4582	-0.5726	0.3868
-0.4673	-0.6576	0.0328	0.4196	-0.1268	0.395
0.2022	-0.0783	-0.7849	-0.1372	0.3589	0.435
-0.3727	0.4665	0.3512	0.0528	0.5732	0.4346
-0.2548	0.0078	0.0787	-0.769	-0.4384	0.3811
0.7165	-0.2725	0.4835	-0.0318	0.0808	0.4135

$\Sigma =$

1.1046	0	0	0	0
0	0.8092	0	0	0
0	0	0.592	0	0
0	0	0	0.1563	0
0	0	0	0	0.0644
0	0	0	0	0

$V =$

-0.3618	0.8667	-0.0353	0.1958	-0.28
-0.3377	-0.4596	0.0405	0.3719	-0.7313
0.6743	0.1088	-0.0908	0.7246	-0.0162
-0.0752	-0.0629	-0.9941	-0.0453	-0.0038
-0.543	-0.1478	0.0232	0.5443	0.6217

PCA Example [2]

- Perform PCA with $k = 2$.

$X =$

feat1	feat2	feat3	feat4	feat5
0.33	-0.20	0.22	0.04	-0.14
-0.26	0.45	-0.36	0.05	0.39
-0.13	-0.09	0.17	0.45	-0.12
0.46	-0.05	-0.25	-0.20	0.20
0.09	0.07	-0.28	-0.02	0.07
-0.49	-0.16	0.48	-0.33	-0.39

$V =$

-0.3618	0.8667	-0.0353	0.1958	-0.28
-0.3377	-0.4596	0.0405	0.3719	-0.7313
0.6743	0.1088	-0.0908	0.7246	-0.0162
-0.0752	-0.0629	-0.9941	-0.0453	-0.0038
-0.543	-0.1478	0.0232	0.5443	0.6217

$Z = XV[:, : 2] =$

feat1	feat2	feat3	feat4	feat5
0.33	-0.20	0.22	0.04	-0.14
-0.26	0.45	-0.36	0.05	0.39
-0.13	-0.09	0.17	0.45	-0.12
0.46	-0.05	-0.25	-0.20	0.20
0.09	0.07	-0.28	-0.02	0.07
-0.49	-0.16	0.48	-0.33	-0.39

-0.3618	0.8667
-0.3377	-0.4596
0.6743	0.1088
-0.0752	-0.0629
-0.543	-0.1478

$=$

New feat1	New feat2
0.1695	0.42
-0.5161	-0.5321
0.2234	-0.0634
-0.4117	0.3775
-0.2815	0.0063
0.7915	-0.2205

PCA Example [3]

- What is the **minimum** number of principal components you would need to explain **at least 95%** of the variance?

$X =$

feat1	feat2	feat3	feat4	feat5
0.33	-0.20	0.22	0.04	-0.14
-0.26	0.45	-0.36	0.05	0.39
-0.13	-0.09	0.17	0.45	-0.12
0.46	-0.05	-0.25	-0.20	0.20
0.09	0.07	-0.28	-0.02	0.07
-0.49	-0.16	0.48	-0.33	-0.39

$\Sigma =$

1.1046	0	0	0	0
0	0.8092	0	0	0
0	0	0.592	0	0
0	0	0	0.1563	0
0	0	0	0	0.0644
0	0	0	0	0

$$\frac{\sum_{j=1}^k \sigma_j^2}{\sum_i \sigma_i^2} \geq 0.95$$

Denominator

$$\sum_i \sigma_i^2 = 2.254$$

With $k = 1$

$$\frac{\sigma_1^2}{\sum_i \sigma_i^2} = 0.54$$

With $k = 2$

$$\frac{\sigma_1^2 + \sigma_2^2}{\sum_i \sigma_i^2} = 0.83$$

With $k = 3$

$$\frac{\sigma_1^2 + \sigma_2^2 + \sigma_3^2}{\sum_i \sigma_i^2} = 0.98$$

Thus, we need $k = 3$ PCs

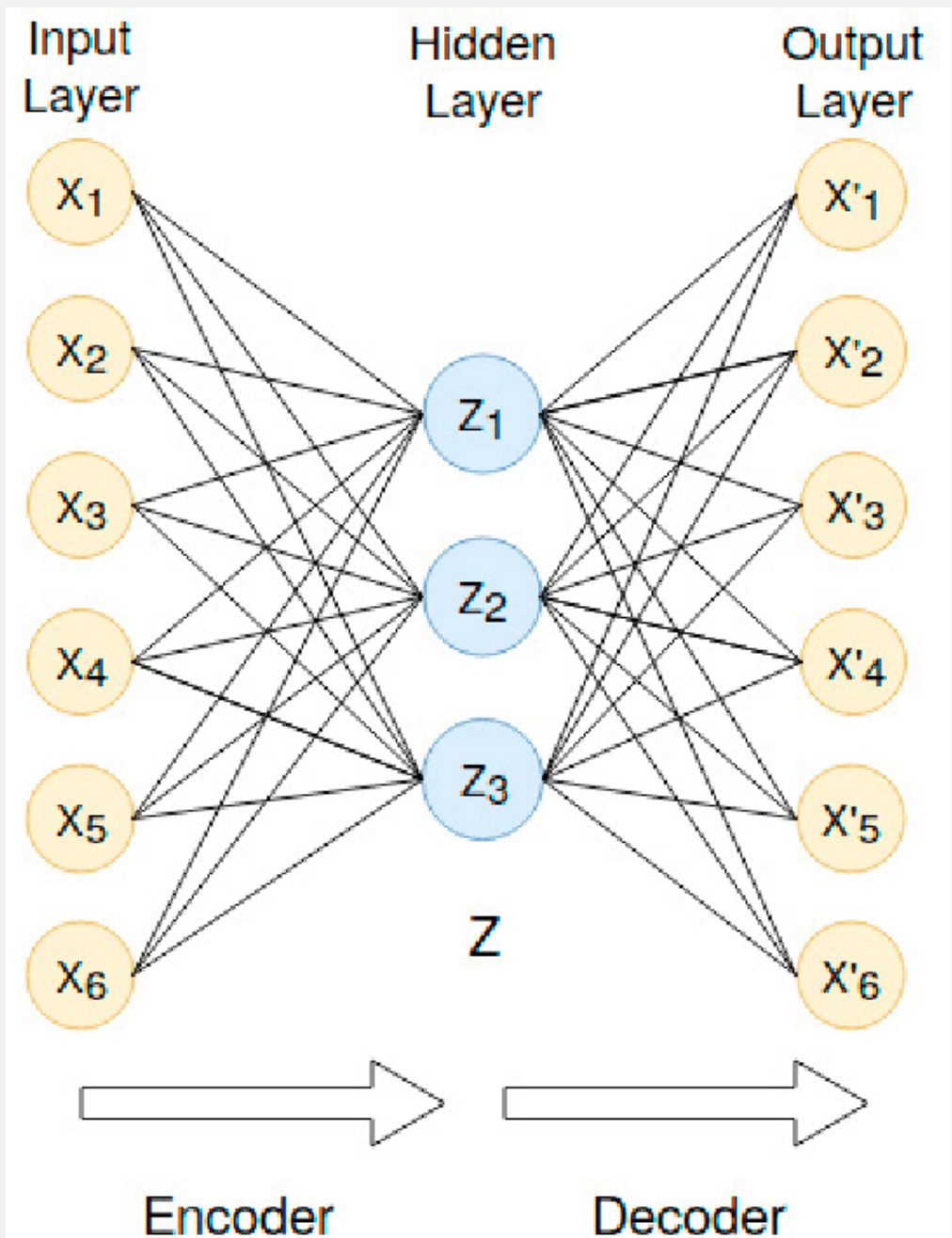
Autoencoders

Autoencoder [1]

- Autoencoders are neural networks where the input layer takes in the data features, and the output layer outputs the data features.
- As a supervised learning task, this means that the output labels are the input features themselves.
- The network is trained to minimize the reconstruction error (e.g. squared error).
- There are many variants of autoencoders. We will just introduce the basic autoencoder.
- Basic Autoencoder: the input layer has n features, the hidden layer has k units, the output layer has n units (features).
- Train the autoencoder to minimize squared error:

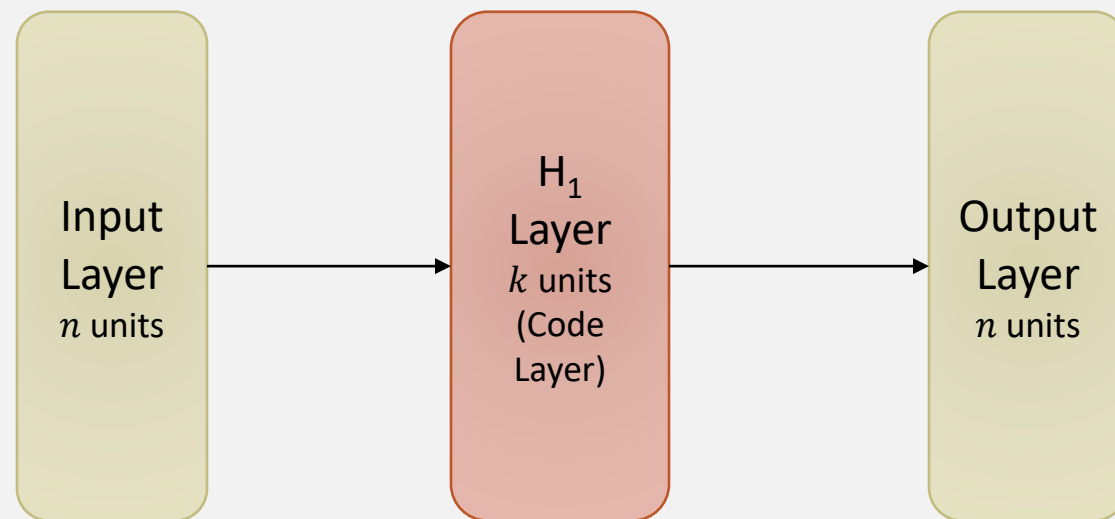
$$\|X - Net(X)\|_2^2$$

Image from: [link](#)



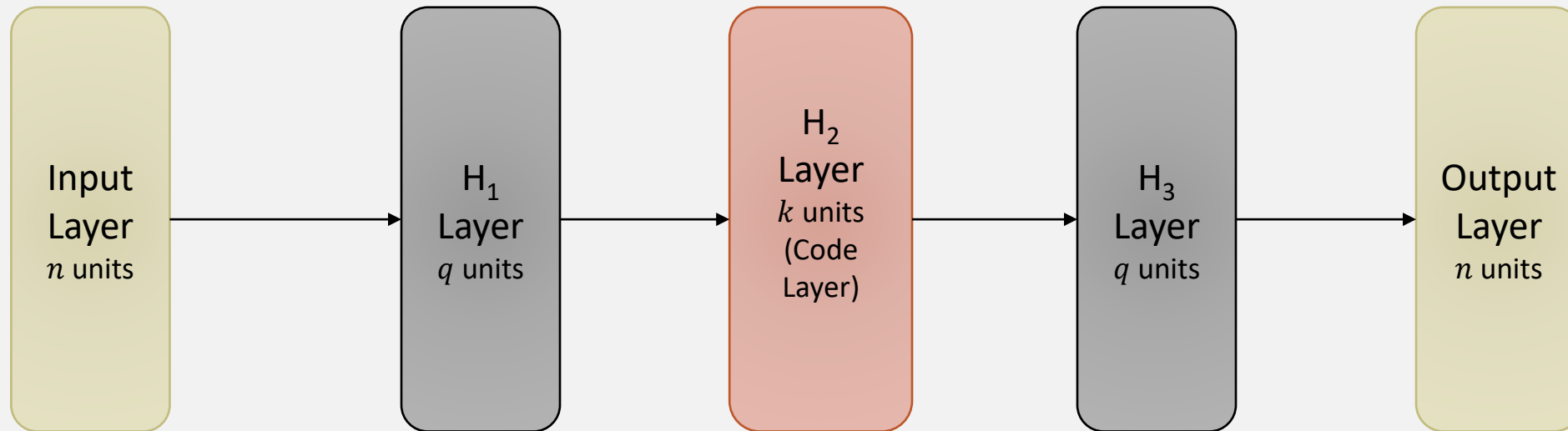
Autoencoder [2]

- If the hidden layer has k units, the autoencoder is learning a k -dimensional representation of the data.
- That is, the autoencoder transforms the data from n dimensions (input layer) to k dimensions (hidden layer), and then back to n dimensions (output layer).
- After training the autoencoder, you can project a datapoint (n dimensions) by passing it to the network and extracting the hidden layer vector (k dimensions). This is sometimes referred to as the code layer.



Autoencoder [3]

- To allow for non-linear transformation, use non-linear activations.
- To allow for even more complex transformations, use more hidden layers.



- Typically, the layers before the encoding layer are collectively called as the *encoder*, and the layers after the encoding layer are collectively called the *decoder*.

Autoencoder [4]

- Using keras, I construct a 1 hidden layer autoencoder. Notice the how fit is called.

```
1 autoencoder_model = keras.Sequential(  
2     [  
3         layers.Input(shape=(n,)), # n is number of features  
4         layers.Dense(k, name="code_layer", activation="sigmoid"), # can use any activation  
5         layers.Dense(n, name="output", activation="linear"),  
6     ]  
7 )  
8  
9 opt = keras.optimizers.Adam(learning_rate=0.01, amsgrad=True)  
0 autoencoder_model.compile(optimizer=opt, loss='mse')  
1  
2 autoencoder_model.fit(X, X, batch_size=512, verbose=2, epochs=1000)
```

- Can extract the coding layer vector for input.

```
1 layer_name = 'code_layer'  
2 coding_layer = keras.Model(inputs=autoencoder_model.input,  
3     outputs=autoencoder_model.get_layer(layer_name).output)  
4 X_transform = coding_layer(X) # we pass X to the trained network and extract the coding layer value
```

t-SNE and UMAP

t-SNE Overview [1]

- t-distributed stochastic neighbor embedding (t-SNE) is a **nonlinear** dimensionality reduction technique. It is mainly used for visualizing high dimensional data.
- The formulation is rather involved, so we will not cover it. But you can read the [Paper \(2008\)](#).
- The *stochastic* in the name comes from how t-SNE finds a low dimensional embedding.
 - It computes the probability that two datapoints are related in the original space.
 - Datapoints that are *similar* (distance) will have high probability.
 - It then finds a low dimensional embedding that preserves this probability as much as possible.

t-SNE Overview [2]

- Implementation in [scikit-learn](#).

```
class sklearn.manifold.TSNE(n_components=2, *, perplexity=30.0, early_exaggeration=12.0, learning_rate='warn', n_iter=1000,
n_iter_without_progress=300, min_grad_norm=1e-07, metric='euclidean', init='warn', verbose=0, random_state=None,
method='barnes_hut', angle=0.5, n_jobs=None, square_distances='legacy')
```

[\[source\]](#)

```
1 from sklearn.manifold import TSNE
2
3 tsne_embedding = TSNE(n_components=2)
4
5 tsne_embedding.fit(X)
6 X_transform = tsne_embedding.transform(X)
```

UMAP Overview [1]

- Uniform manifold approximation and projection (UMAP) is a **nonlinear** dimensionality reduction technique. Paper released around 2018.
- The formulation is rather involved, so we will not cover it. But you can read the original [paper](#) and see the Github repository for the package [\[link\]](#).
- In *very simple* terms, UMAP finds a low dimensional projection of the data that has the closest possible structure of the original data.
- Since its release it has become a popular method to visualize high dimensional data.
- Has a number of parameters that affect the results. See the [documentation](#).
- Many visualization examples on various datasets can be found at the package website.

UMAP Overview [2]

- The package is called [umap-learn](#). It is easy to use:

```
1 import umap
2
3 umap_embedding = umap.UMAP(n_neighbors=5, min_dist=0.3, metric='jaccard')
4
5 umap_embedding.fit(X)
6 X_transform = umap_embedding.transform(X)
```

UMAP Example with MNIST data

- MNIST is a dataset of number images (8x8 pixels or if flattened is 64 feature vector).
- The following is UMAP reduction into 2 dimensions.

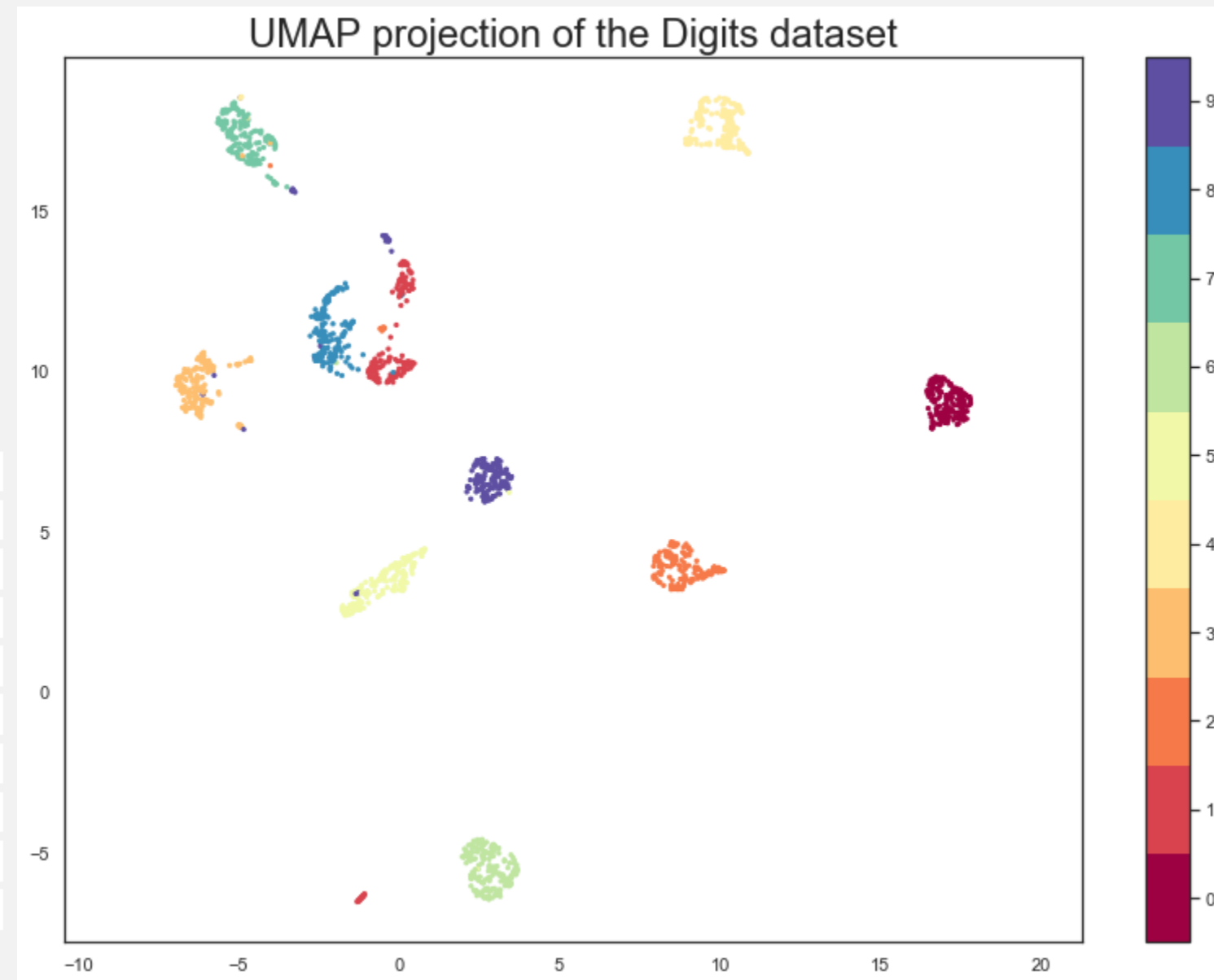
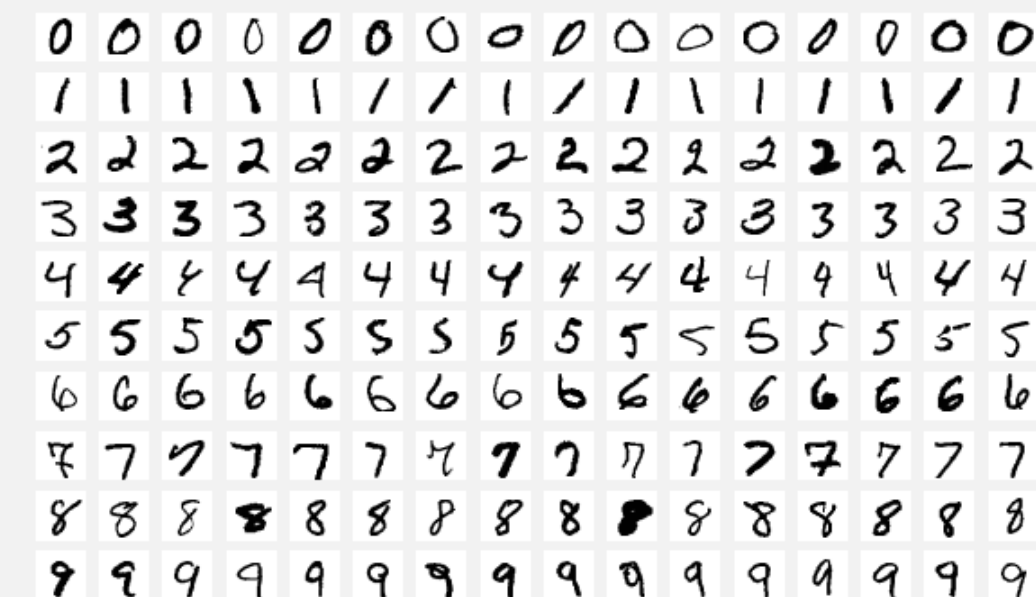
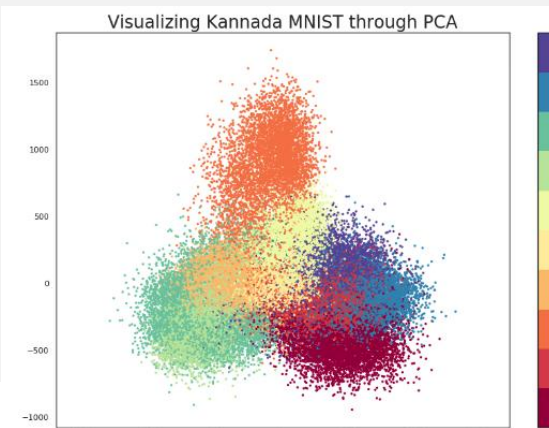
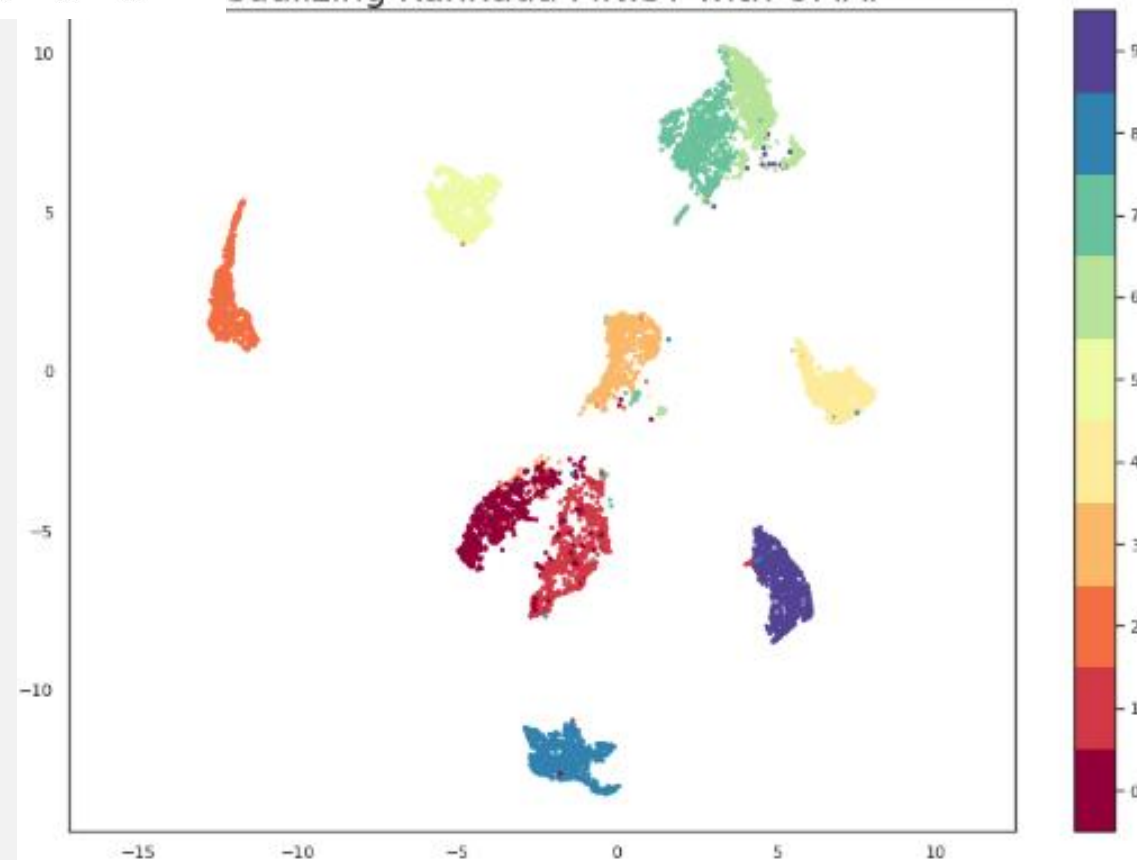


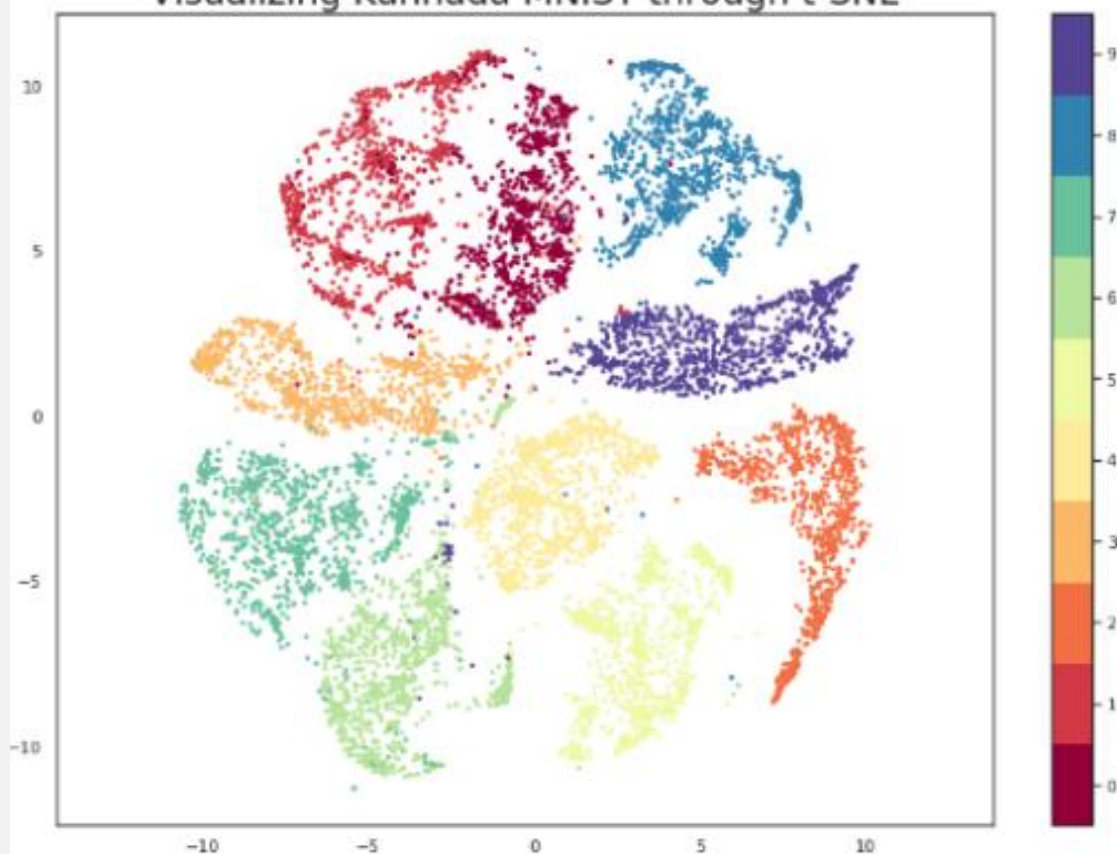
Image from: [link](#)



Visualizing Kannada MNIST with UMAP



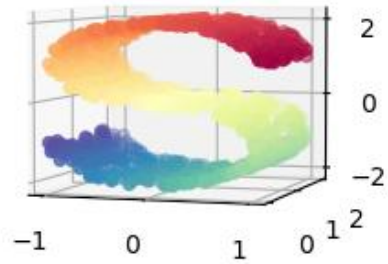
Visualizing Kannada MNIST through t-SNE



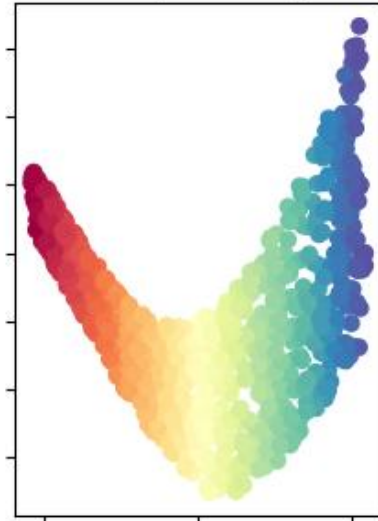
- Other methods for dimensionality reduction implemented in scikit-learn.

<code>manifold.Isomap(*[, n_neighbors, ...])</code>	Isomap Embedding.
<code>manifold.LocallyLinearEmbedding(*[, ...])</code>	Locally Linear Embedding.
<code>manifold.MDS([n_components, metric, n_init, ...])</code>	Multidimensional scaling.
<code>manifold.SpectralEmbedding([n_components, ...])</code>	Spectral embedding for non-linear dimensionality reduction.
<code>manifold.TSNE([n_components, perplexity, ...])</code>	T-distributed Stochastic Neighbor Embedding.
<code>manifold.locally_linear_embedding(X, *, ...)</code>	Perform a Locally Linear Embedding analysis on the data.
<code>manifold.smacof(dissimilarities, *[, ...])</code>	Compute multidimensional scaling using the SMACOF algorithm.
<code>manifold.spectral_embedding(adjacency, *[, ...])</code>	Project the sample on the first eigenvectors of the graph Laplacian.
<code>manifold.trustworthiness(X, X_embedded, *[, ...])</code>	Expresses to what extent the local structure is retained.

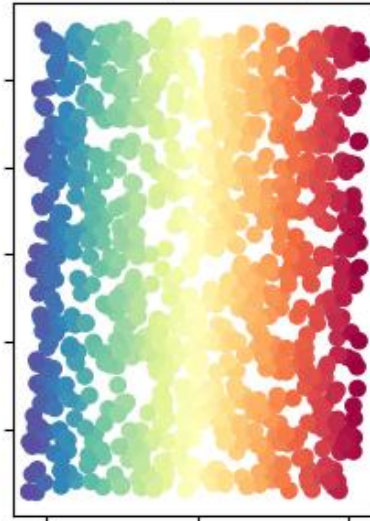
Manifold Learning with 1000 points, 10 neighbors



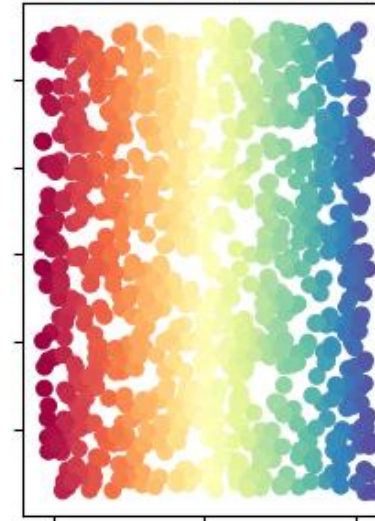
LLE (0.12 sec)



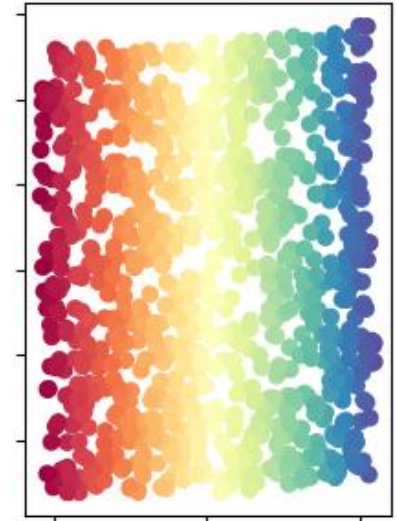
LTSA (0.21 sec)



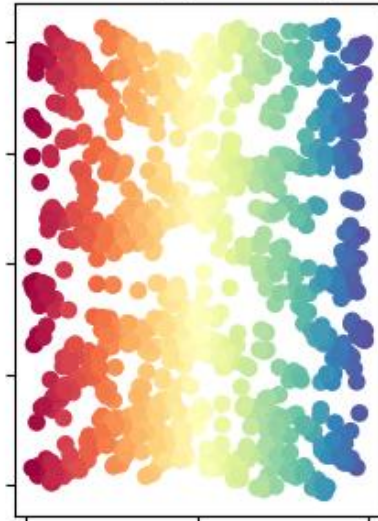
Hessian LLE (0.32 sec)



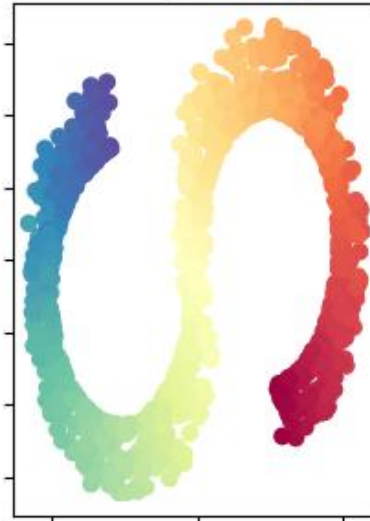
Modified LLE (0.26 sec)



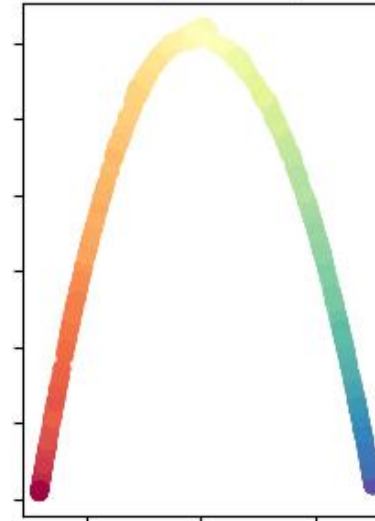
Isomap (0.72 sec)



MDS (2.2 sec)



SE (0.12 sec)



t-SNE (11 sec)

